

Layer-Adaptive XMSSMT Parameterization: A Multi-Objective Study of WOTS Placement, Tree-Height Allocation, and Serialized State

Sohaib Rana¹, Mohd Najwadi Yusoff^{1,2*}, Je Sen Teh^{3,4*},
Adnan Anwar³

¹School of Computer Sciences, Universiti Sains Malaysia, Gelugor, Penang, Malaysia.

²Cybersecurity Research Centre (CYRES), Universiti Sains Malaysia, Gelugor, Penang, Malaysia.

³School of Information Technology, Deakin University, Waurin Ponds, Victoria, Australia.

⁴Deakin Cyber, Deakin University, Waurin Ponds, Victoria, Australia.

*Corresponding author(s). E-mail(s): najwadi@usm.my;
j.teh@deakin.edu.au;

Contributing authors: sohaibrana2023@student.usm.my;
adnan.anwar@deakin.edu.au;

Abstract

XMSSMT implementations typically use uniform per-layer parameterization, although the computational and state-management roles of individual layers are asymmetric. This study investigates layer-adaptive XMSSMT parameterization by independently assigning Winternitz parameters and tree heights across a two-layer XMSSMT construction with total height $h = 20$. Five height allocations, $(10, 10)$, $(8, 12)$, $(12, 8)$, $(6, 14)$, and $(14, 6)$, were combined with four WOTS allocations, $(16, 16)$, $(4, 16)$, $(16, 4)$, and $(4, 4)$, producing a 20-configuration design space. Experiments were conducted on a Raspberry Pi 5 using a FAST BDS implementation with fixed CPU affinity and controlled benchmarking. Across configurations with identical 7075-B signatures, placing $w = 4$ in the lower layer reduced mean signing latency by 40.83% to 45.09% relative to upper-layer placement and reduced serialized secret-key state by 2112 B. Redistributing tree height changed serialized state systematically by ± 240 B or ± 480 B relative to the balanced $(10, 10)$ allocation, while latency

effects were nonlinear and operation-dependent. Multi-objective analysis over key-generation latency, signing latency, verification latency, signature size, and serialized secret-key size identified 13 nondominated configurations among the 20 tested settings. The results show that XMSSMT layer parameters should be treated as ordered, deployment-dependent design variables rather than interchangeable homogeneous settings. The evaluated configurations are experimental and are not standardized XMSSMT parameter sets.

Keywords: XMSSMT, XMSS, hash-based signatures, post-quantum cryptography, WOTS+, Winternitz parameter, BDS traversal, heterogeneous parameterization, Raspberry Pi 5, multi-objective optimization

1 Introduction

1.1 Motivation

Hash-based signatures provide a conservative approach to post-quantum digital signatures because their security can be based primarily on properties of cryptographic hash functions. XMSS and its multi-tree extension XMSSMT are stateful hash-based signature schemes designed for applications in which a bounded number of signatures can be generated under carefully managed signing state.

XMSSMT organizes multiple XMSS trees into a hierarchy. In the standardized XMSSMT construction, the total tree height h is divided uniformly among d layers, so that every layer has height h/d , and the same Winternitz parameter w is used throughout the hierarchy [1]. NIST SP 800-208 similarly specifies approved XMSSMT parameter sets using fixed parameter combinations, including $w=16$ for the approved XMSSMT families [2].

Uniform parameterization simplifies specification, interoperability, and implementation. However, the layers of XMSSMT do not perform identical computational roles. The lowest layer directly signs messages, whereas higher layers authenticate roots of lower-layer trees. In implementations using BDS traversal, individual layers can also contribute differently to maintained and serialized state. These structural asymmetries motivate studying whether parameter placement across layers changes practical performance even when the overall XMSSMT height and cryptographic building blocks remain fixed.

This question is particularly relevant to constrained and embedded deployments, where signing latency, key-generation cost, signature size, and persistent state can all affect feasibility. A parameter assignment that improves one metric may impose substantial penalties on another. XMSSMT parameter selection is therefore naturally a multi-objective implementation problem.

1.2 Research gap

Layer-specific XMSSMT parameters are not themselves a new theoretical concept. Prior work on optimal XMSSMT parameters modeled one parameter set per layer,

including layer-specific tree heights h_i , BDS parameters k_i , and Winternitz parameters w_i [3]. Consequently, heterogeneous XMSSMT must not be framed as being introduced for the first time in this work.

The remaining gap is empirical and implementation-oriented. In particular, the practical consequences of ordered WOTS placement deserve direct measurement. Assignments such as $(w_0, w_1) = (4, 16)$ and $(16, 4)$ contain the same Winternitz values and can produce the same signature size, but the two assignments need not incur the same signing or serialized-state cost because the two XMSSMT layers serve different roles.

A related question concerns asymmetric tree-height allocation. For a fixed total height, redistributing tree levels between the lower and upper layers changes the topology and size of the traversal state. The resulting effects on key generation, signing, verification, and serialized state are not necessarily monotonic.

Existing theoretical parameter optimization motivates an implementation-level investigation of ordered per-layer choices on an embedded platform.

1.3 Research question

This study addresses the following research question:

How do layer-specific Winternitz parameters and tree-height allocations affect key-generation latency, signing latency, verification latency, signature size, and serialized state in a two-layer XMSSMT FAST/BDS implementation?

To investigate this question, the study fixes the total XMSSMT height at $h=20$ and evaluates five ordered layer-height allocations:

$(10, 10)$, $(8, 12)$, $(12, 8)$, $(6, 14)$, and $(14, 6)$.

These are combined with four ordered WOTS allocations:

$(16, 16)$, $(4, 16)$, $(16, 4)$, and $(4, 4)$.

The resulting design space contains 20 configurations. The configurations are experimental implementation-level parameterizations and are not standardized XMSSMT parameter sets or standardized OIDs.

1.4 Contributions

The main contributions of this work are as follows.

First, we implement independent tree heights and Winternitz parameters for each layer in a two-layer XMSSMT FAST/BDS implementation with fixed total height.

Second, we conduct an operation-level study of 20 combinations of height allocation and ordered WOTS placement on Raspberry Pi 5, with correctness validation and controlled key-generation, signing, and verification measurements.

Third, we isolate the effect of WOTS placement by comparing $(4, 16)$ and $(16, 4)$. Although both configurations produce 7075-byte signatures in the evaluated design, placing $w=4$ in the lower layer reduces mean signing latency by 40.83% to 45.09% across the five tested height allocations and reduces serialized secret-key state by 2112 bytes in every comparison.

Fourth, we quantify the effect of asymmetric height allocation on serialized BDS state. Relative to the balanced $(10, 10)$ allocation, the tested configurations change

serialized secret-key size by -240, +240, -480, or +480 bytes, while their latency effects remain nonlinear and operation-dependent.

Fifth, we perform a five-objective empirical Pareto analysis over key-generation latency, signing latency, verification latency, signature size, and serialized secret-key size. Thirteen of the 20 tested configurations are nondominated, illustrating that the preferred heterogeneous parameterization depends on deployment priorities rather than on a single universal optimum.

The study therefore does not claim to introduce heterogeneous XMSSMT parameterization as a new theoretical concept. Instead, it provides an implementation-level empirical characterization of ordered layer-specific parameter placement, asymmetric traversal state, and their resulting deployment trade-offs.

2 Background and Related Work

2.1 XMSS and XMSSMT

XMSS is a stateful hash-based signature construction in which one-time WOTS+ key pairs are organized as leaves of a Merkle tree. Each signature contains a WOTS+ signature together with an authentication path allowing the verifier to reconstruct the public Merkle root.

XMSSMT extends this construction by arranging multiple XMSS trees in a hierarchical multi-tree structure. Trees at the lowest layer sign actual messages, while trees at higher layers authenticate root nodes of trees below them. In RFC 8391, an XMSSMT construction with total height h and d layers uses trees of equal height h/d at every layer and the same Winternitz parameter w throughout the hierarchy [1].

The number of layers d provides a fundamental trade-off. Increasing d generally reduces individual subtree height and can improve key-generation and signing behavior, but increases signature size because an XMSSMT signature contains one WOTS+ signature and authentication path contribution for each layer.

2.2 WOTS+ parameterization

WOTS+ is the one-time signature primitive used within XMSS and XMSSMT. Its Winternitz parameter w determines a trade-off between signature length and chain-computation cost. Smaller w values require more WOTS+ chains but shorter average chain traversal, whereas larger values reduce the number of signature elements while increasing chain length.

RFC 8391 permits w values including 4 and 16 at the WOTS+ level, while its specified XMSS and XMSSMT parameter sets use $w=16$ for efficiency and implementation simplicity [1].

In standardized XMSSMT, the same w is assigned to all layers. This uniform assignment does not expose whether the same Winternitz value has an equivalent implementation cost when placed in different layers of the hierarchy.

2.3 BDS traversal and state costs

Merkle-tree authentication paths can be recomputed directly for each signature, but this approach has high computational cost. The BDS traversal algorithm reduces signing cost by maintaining intermediate tree state between signatures [4].

The FAST XMSS reference implementation applies BDS traversal and therefore exchanges additional persistent state for substantially faster signature generation. Prior embedded XMSS work has explicitly contrasted the straightforward recomputation implementation with the BDS-based FAST implementation, noting that the latter requires additional secret-key state but provides more practical signing performance [5].

This traversal structure is important for heterogeneous XMSSMT because each layer’s serialized state depends on its tree height and on whether current and next subtree states are retained. Consequently, redistributing height across layers can affect serialized state even when the total XMSSMT height remains unchanged.

2.4 Implementation and optimization studies

A substantial body of XMSS implementation research has focused on reducing computational cost through optimized hashing, precomputation, alternative hash constructions, and platform-specific acceleration.

Campos et al. compared LMS and XMSS on ARM Cortex-M4 and evaluated several XMSS and XMSSMT implementation variants, including simplified hashing and precomputation. Their results demonstrate that implementation choices can materially alter key-generation and signing cost on constrained ARM platforms [6].

Other work has examined accelerated XMSS verification, including WOTS+-oriented optimizations and ARM Cortex-M4 implementations [7]. Hardware-oriented studies have also explored configurable XMSS accelerators that vary hash cores, WOTS+ chain units, tree height, and Winternitz parameters to expose performance/resource trade-offs [8].

These studies show that XMSS performance is implementation- and platform-dependent. However, most implementation evaluations retain homogeneous XMSSMT layer parameters rather than directly comparing ordered WOTS assignments such as (4,16) and (16,4).

2.5 Prior XMSSMT parameter optimization

Heterogeneous layer parameters are not a new theoretical idea. Prior work on optimal XMSSMT parameters formulated parameter selection at the layer level and included per-layer tree heights h_i , BDS parameters k_i , and Winternitz parameters w_i [3].

That work addressed parameter optimization analytically and demonstrated that XMSSMT exposes a large design space involving runtimes, signature sizes, and traversal parameters. It therefore provides an important theoretical precedent for layer-specific XMSSMT optimization.

The present study is positioned differently. Rather than claiming the first heterogeneous XMSSMT construction, it operationalizes layer-specific parameterization in

a FAST/BDS implementation and evaluates the resulting effects experimentally on Raspberry Pi 5.

2.6 Gap addressed in this study

The central gap addressed here is the empirical effect of parameter placement.

First, the study directly compares the ordered WOTS allocations (4,16) and (16,4). These assignments contain the same two Winternitz values and produce equal signature sizes in the evaluated two-layer construction, but their computational and serialized-state costs may differ because the two layers perform asymmetric roles.

Second, the study evaluates asymmetric height allocations while keeping total XMSSMT height fixed. This isolates how the distribution of traversal state between lower and upper layers changes serialized secret-key size and operation latency.

Third, the study jointly evaluates these two dimensions across a 20-configuration design space and applies a five-objective empirical Pareto analysis. The resulting framework connects theoretical parameter flexibility with measured deployment-level trade-offs in signing latency, key-generation latency, verification latency, signature size, and persistent state.

Accordingly, the contribution is an implementation-level empirical characterization of ordered layer-specific XMSSMT parameterization rather than a new theoretical introduction of heterogeneous XMSSMT parameters.

3 Methods

3.1 Experimental platform

All experiments were executed on a Raspberry Pi 5 Model B Rev 1.1 using a 64-bit ARM environment. The platform used the ARM Cortex-A76 processor architecture with four logical CPUs. The operating system was Debian 13 (trixie), and the implementation was compiled with GCC 14.2.0.

To reduce scheduling variability, publication measurements were executed with CPU affinity fixed to logical CPU 3 using taskset. The CPU-frequency governor was configured as performance, processor boost was disabled, and the platform reported a maximum CPU frequency of 2400 MHz and a minimum frequency of 1500 MHz.

Timing measurements used `CLOCK_MONOTONIC_RAW`. Platform throttling status was checked using `vcgencmd get_throttled`, and authoritative runs were accepted only when no throttling condition was observed.

3.2 Implementation and compilation

The experimental implementation was derived from the frozen XMSS/XMSSMT working baseline used in Objective 1. The heterogeneous extension modified XMSSMT parameter handling to permit layer-specific Winternitz parameters and layer-specific tree heights while preserving the total XMSSMT height.

The authoritative compilation configuration used:

```
-I. -std=gnu11 -O3 -Wall -Wextra -Wpedantic -DNDEBUG -DXMSSMT
-DFAST_IMPL
```

The FAST implementation used BDS traversal state. The baseline parameter structure was extended with per-layer WOTS and tree-height fields, and helper routines were introduced to derive layer-specific WOTS parameters, serialized state offsets, BDS-state sizes, and per-layer tree-height metadata.

3.3 Heterogeneous XMSSMT parameterization

The study considered a two-layer XMSSMT construction with total tree height $h=20$. A heterogeneous parameter configuration was represented as

$\Theta = (h_0, w_0), (h_1, w_1)$

subject to

$h_0 + h_1 = 20$.

Five layer-height allocations were evaluated:

(10,10), (8,12), (12,8), (6,14), and (14,6).

Four WOTS allocations were evaluated:

(16,16), (4,16), (16,4), and (4,4).

The Cartesian product therefore produced 20 configurations.

The homogeneous configuration (10,10) with (16,16) served as the principal structural control. Heterogeneous configurations were experimental parameterizations and were not represented as standardized XMSSMT OIDs.

3.4 Correctness validation

Correctness testing preceded authoritative benchmarking. Each new heterogeneous configuration was exercised using fresh keys and sequential signatures with verification after signing.

Targeted rollover tests were additionally performed for height allocations with short lower-layer trees. In particular, configurations using the (6,14) height allocation were exercised across the lower-layer transition from signature index 63 to 64. All accepted correctness runs preserved the expected signature, public-key, and serialized secret-key sizes and successfully verified the generated signatures.

3.5 Benchmark protocol

The authoritative benchmark protocol for each configuration comprised:

10 key-generation measurements, 100 signing measurements, and 100 verification measurements.

This produced 210 operation-level measurements per configuration and 4200 normalized measurements across the 20-cell joint design space.

Warm-up execution used three key generations and 100 signatures before authoritative measurement. Timing results were recorded in nanoseconds and converted to milliseconds for analysis.

For the H6-14 configurations, the 100-signature sequence includes the lower-layer rollover at signature indices 63 to 64. Consequently, the reported mean signing latency for these configurations includes this rollover event.

3.6 Artifact-size accounting

The study recorded signature size, public-key size, and serialized secret-key size.

For a fixed WOTS allocation, redistributing the total height between the two XMSSMT layers did not alter signature size because the aggregate authentication-path height remained 20.

Serialized secret-key size depended on the layer-specific BDS states and retained upper-layer WOTS state. For the tested two-layer FAST structure, changing the height allocation from (10,10) produced serialized-state changes of -240 B for (8,12), +240 B for (12,8), -480 B for (6,14), and +480 B for (14,6).

Placement of $w=4$ in the upper layer additionally increased retained WOTS state relative to lower-layer placement. This produced a 2112-B serialized secret-key difference between W4-16 and W16-4 at every tested height allocation.

3.7 Joint dataset construction

The 20-cell analysis reused measurements from three previously frozen experimental phases:

HEIGHT, WOTS, and JOINT.

Five homogeneous-WOTS height-allocation configurations were taken from the frozen height experiment, three WOTS-placement configurations at the balanced height split were taken from the frozen WOTS experiment, and twelve remaining heterogeneous combinations were measured in the joint experiment.

The resulting source map contained exactly 20 unique parameter configurations.

All source datasets were transformed into a common normalized schema containing configuration identifiers, layer heights, layer-specific WOTS parameters, operation type, iteration number, latency, verification result, and artifact sizes.

The normalization gate verified exactly 4200 operation-level rows: 20 configurations multiplied by 210 measurements per configuration.

3.8 Descriptive and Pareto analysis

For each configuration and operation type, descriptive statistics were derived from the normalized measurements. Mean key-generation, signing, and verification latencies were used in the paper-facing joint summary.

Multi-objective comparison minimized five objectives:

key-generation latency, signing latency, verification latency, signature size, and serialized secret-key size.

Public-key size was excluded from the Pareto objectives because it was constant at 64 B across the evaluated configurations.

A configuration was classified as dominated when at least one other configuration was no worse in all five objectives and strictly better in at least one objective. Thirteen of the 20 configurations were nondominated and seven were strictly dominated.

Because the 20-cell dataset combines measurements from previously frozen HEIGHT, WOTS, and JOINT phases rather than a single session-matched factorial experiment, the Pareto analysis is interpreted as an empirical comparison of frozen datasets.

3.9 Reproducibility and provenance

Correctness outputs, benchmark datasets, normalization scripts, analysis scripts, tables, manifests, and SHA-256 records were preserved in immutable freeze directories.

The joint analysis freeze was closed with 30 manifest entries and passed final SHA-256 integrity verification. Paper-facing tables and figures were subsequently generated outside the immutable experimental freeze to prevent modification of authoritative measurement artifacts.

4 Results

4.1 Joint heterogeneous parameter landscape

The joint parameter study evaluated 20 heterogeneous XMSSMT configurations obtained from the Cartesian combination of five layer-height allocations, (10,10), (8,12), (12,8), (6,14), and (14,6), and four layer-specific WOTS allocations, (16,16), (4,16), (16,4), and (4,4). Each configuration was represented by 10 key-generation measurements, 100 signing measurements, and 100 verification measurements, yielding 4,200 normalized operation-level observations across the complete design space.

The measured configurations exhibited substantial variation in operation latency and serialized artifact size. The minimum observed key-generation latency was 2756.037 ms for H10-10_W4-4, whereas the minimum signing latency was 4.302 ms for H10-10_W4-16. The minimum verification latency was 1.639 ms for H6-14_W4-4. The smallest signature size was 4963 B and occurred for all configurations using the homogeneous WOTS allocation W16-16. The minimum serialized secret-key size was 5518 B, observed for H6-14.W16-16 and H6-14_W4-16.

These extrema did not occur in a single configuration, indicating that heterogeneous layer parameters induce distinct trade-offs among key generation, signing, verification, signature size, and serialized state.

Table 1 summarizes the complete 20-configuration design space. Figures 1 and 2 visualize the relationships between signing latency, signature size, and serialized secret-key size.

4.2 Effect of WOTS placement

A direct comparison between W4-16 and W16-4 isolates the effect of placing the smaller Winternitz parameter in the lower or upper XMSSMT layer. These two parameterizations produce the same 7075-B signature size, allowing the placement effect to be examined without a signature-size confound.

Across all five tested height allocations, assigning $w=4$ to the lower layer reduced signing latency relative to assigning $w=4$ to the upper layer. The reduction ranged from 40.83% to 45.09%. For the (10,10) height allocation, signing latency decreased from 7.830 ms for W16-4 to 4.302 ms for W4-16, corresponding to a 45.06% reduction. The corresponding reductions were 40.83% for (8,12), 43.88% for (12,8), 45.09% for (6,14), and 44.75% for (14,6).

The placement effect was also reflected in serialized secret-key size. At every tested height allocation, the W4-16 configuration required 2112 B less serialized secret-key

Table 1 Mean operation latency and artifact sizes for the 20 evaluated heterogeneous XMSSMT configurations. Each configuration contributes 10 key-generation, 100 signing, and 100 verification measurements. Source identifies the frozen experimental phase from which the configuration was obtained.

Config.	h_0	h_1	w_0	w_1	KeyGen (ms)	Sign (ms)	Verify (ms)	Sig. (B)	SK (B)	Source
H10-10_W16-16	10	10	16	16	4888.614	11.029	2.611	4963	5998	HEIGHT
H10-10_W16-4	10	10	16	4	3820.982	7.830	2.143	7075	8110	WOTS
H10-10_W4-16	10	10	4	16	3829.627	4.302	2.136	7075	5998	WOTS
H10-10_W4-4	10	10	4	4	2756.037	4.306	1.700	9187	8110	WOTS
H12-8_W16-16	12	8	16	16	10411.905	11.165	2.600	4963	6238	HEIGHT
H12-8_W16-4	12	8	16	4	10100.190	11.183	2.147	7075	8350	JOINT
H12-8_W4-16	12	8	4	16	6222.789	6.276	2.159	7075	6238	JOINT
H12-8_W4-4	12	8	4	4	5827.106	6.165	1.696	9187	8350	JOINT
H14-6_W16-16	14	6	16	16	39879.916	11.454	2.783	4963	6478	HEIGHT
H14-6_W16-4	14	6	16	4	39170.225	11.276	2.152	7075	8590	JOINT
H14-6_W4-16	14	6	4	16	22103.821	6.230	2.031	7075	6478	JOINT
H14-6_W4-4	14	6	4	4	22052.330	6.245	1.708	9187	8590	JOINT
H6-14_W16-16	6	14	16	16	39277.915	8.868	2.701	4963	5518	HEIGHT
H6-14_W16-4	6	14	16	4	22314.663	8.640	2.183	7075	7630	JOINT
H6-14_W4-16	6	14	4	16	39239.670	4.744	2.174	7075	5518	JOINT
H6-14_W4-4	6	14	4	4	22051.039	4.736	1.639	9187	7630	JOINT
H8-12_W16-16	8	12	16	16	11501.286	12.068	2.822	4963	5758	HEIGHT
H8-12_W16-4	8	12	16	4	6279.254	11.264	2.217	7075	7870	JOINT
H8-12_W4-16	8	12	4	16	11193.323	6.665	2.234	7075	5758	JOINT
H8-12_W4-4	8	12	4	4	5840.800	6.030	1.645	9187	7870	JOINT

Table 2 Effect of WOTS placement for configurations containing one $w = 4$ and one $w = 16$ layer. Lower-layer placement of $w = 4$ is compared with upper-layer placement at each height allocation.

Height	Lower $w = 4$ Sign (ms)	Upper $w = 4$ Sign (ms)	Reduction (%)	Sig. (B)	Lower SK (B)	Upper SK (B)	SK saving (B)
10-10	4.302	7.830	45.06	7075	5998	8110	2112
8-12	6.665	11.264	40.83	7075	5758	7870	2112
12-8	6.276	11.183	43.88	7075	6238	8350	2112
6-14	4.744	8.640	45.09	7075	5518	7630	2112
14-6	6.230	11.276	44.75	7075	6478	8590	2112

storage than the corresponding W16-4 configuration. For example, at (10,10), the serialized secret key decreased from 8110 B for W16-4 to 5998 B for W4-16. This 2112-B difference was invariant across all five height allocations.

These results show that, within the tested implementation, lower-layer placement of $w=4$ provides a consistent signing-latency advantage and avoids the additional retained WOTS state associated with upper-layer placement.

The complete placement comparison is reported in Table 2 and visualized in Figure 3.

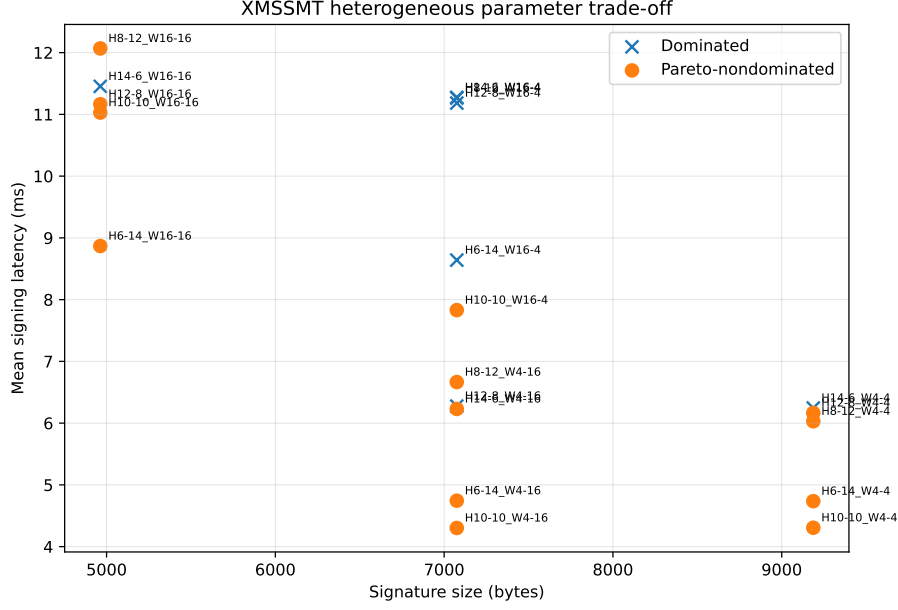


Fig. 1 Mean signing latency as a function of signature size across the evaluated heterogeneous XMSSMT configurations.

4.3 Effect of layer-height allocation

Changing the distribution of the total XMSSMT tree height between the two layers affected serialized BDS state and operation latency while leaving signature size unchanged for a fixed WOTS allocation.

Relative to the balanced (10,10) allocation, the (8,12) allocation reduced serialized secret-key size by 240 B, whereas (12,8) increased it by 240 B. More extreme redistribution produced proportional changes: (6,14) reduced the serialized secret key by 480 B, while (14,6) increased it by 480 B.

The timing effects were not monotonic. Under W16-16, for example, H6-14_W16-16 reduced signing latency from 11.029 ms for H10-10_W16-16 to 8.868 ms, corresponding to a reduction of approximately 19.60%. However, key-generation latency increased from 4888.614 ms to 39277.915 ms. Therefore, height redistribution can reduce serialized state and, in some configurations, signing latency, but may incur a substantial key-generation cost.

This behavior was configuration-dependent across the four WOTS allocations. Consequently, layer-height allocation should not be interpreted as producing a uniformly monotonic latency effect.

Table 3 reports the complete height-allocation comparison. Figures 4 and 5 show the corresponding key-generation and signing trends.

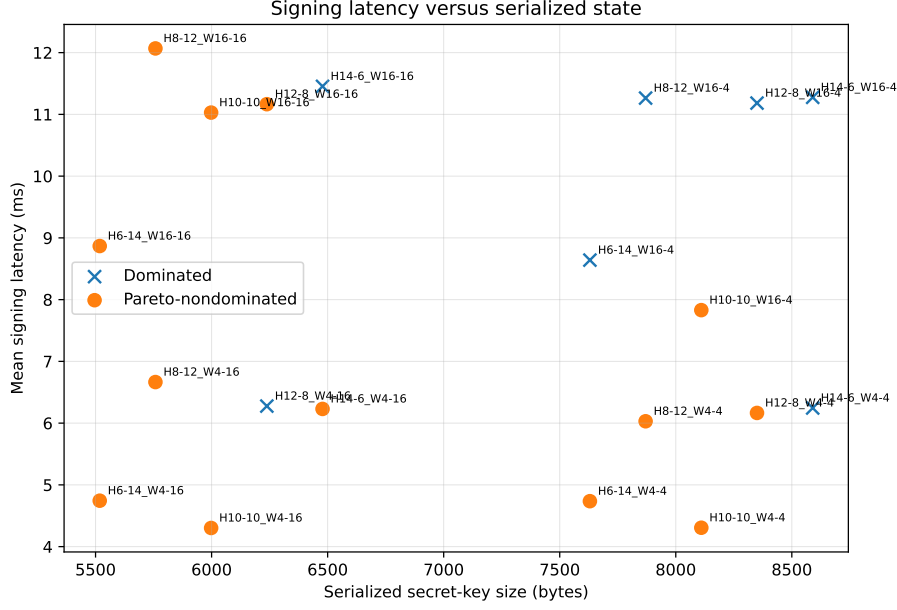


Fig. 2 Mean signing latency as a function of serialized secret-key size across the evaluated heterogeneous XMSSMT configurations.

4.4 Multi-objective Pareto analysis

The 20 configurations were evaluated using five minimized objectives: key-generation latency, signing latency, verification latency, signature size, and serialized secret-key size. Thirteen configurations were nondominated and seven were strictly dominated.

The dominated configurations were H12-8_W16-4, H12-8_W4-16, H14-6_W16-16, H14-6_W16-4, H14-6_W4-4, H6-14_W16-4, and H8-12_W16-4. The remaining 13 configurations formed the empirical Pareto set.

The size of the Pareto set reflects the multi-objective nature of the design space. No single configuration simultaneously minimized all five objectives. H10-10_W4-16 achieved the minimum signing latency, H10-10_W4-4 achieved the minimum key-generation latency, H6-14_W4-4 achieved the minimum verification latency, the W16-16 configurations achieved the minimum signature size, and H6-14_W16-16 together with H6-14_W4-16 achieved the minimum serialized secret-key size.

Accordingly, the results support deployment-dependent parameter selection rather than a universal optimum. Configurations emphasizing signing latency, signature size, serialized state, or initialization cost occupy different regions of the empirical Pareto surface.

The 13 nondominated configurations are listed in Table 4.

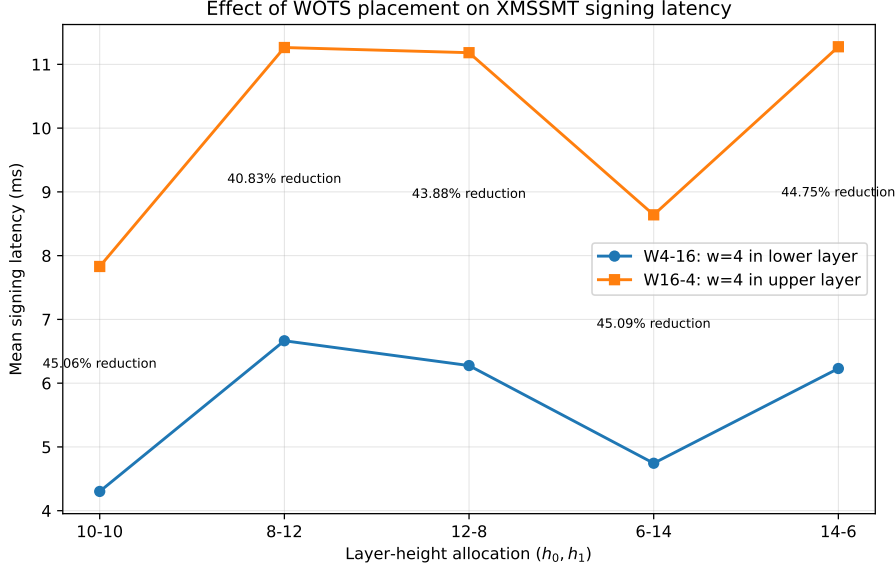


Fig. 3 Mean signing latency for lower-layer and upper-layer placement of $w = 4$ across the five evaluated height allocations.

4.5 Interpretation constraints

The heterogeneous parameterizations evaluated in this study are experimental configurations and should not be interpreted as standardized XMSSMT parameter sets or standardized OIDs.

The 20-cell joint analysis combines frozen measurements obtained from the HEIGHT, WOTS, and JOINT experimental phases. The resulting Pareto analysis should therefore be interpreted as an empirical comparison of frozen datasets rather than as a fully session-matched factorial experiment.

For the H6-14 configurations, the 100-signature measurement sequence crosses the lower-layer rollover boundary at signature indices 63 to 64. This should be considered when interpreting the corresponding mean signing latency.

The results therefore characterize the tested Raspberry Pi 5 implementation and parameter grid and should not be generalized beyond the evaluated implementation, platform, and measurement protocol without additional validation.

5 Discussion

5.1 Implications of layer-specific WOTS placement

The clearest empirical result of the heterogeneous XMSSMT study is that the placement of the smaller Winternitz parameter matters independently of signature size. The W4-16 and W16-4 configurations both generate 7075-B signatures, yet their signing behavior differs substantially across every tested layer-height allocation. Placing

Table 3 Effect of layer-height allocation within each WOTS class. Percentage and serialized-state changes are reported relative to the balanced (10, 10) height allocation for the corresponding WOTS pair.

WOTS	Height	KeyGen (ms)	Sign (ms)	Verify (ms)	Sig. (B)	SK (B)	Δ KeyGen (%)	Δ Sign (%)	Δ SK (B)
W16-16	10-10	4888.614	11.029	2.611	4963	5998	0.00	0.00	0
W16-16	8-12	11501.286	12.068	2.822	4963	5758	135.27	9.42	-240
W16-16	12-8	10411.905	11.165	2.600	4963	6238	112.98	1.24	240
W16-16	6-14	39277.915	8.868	2.701	4963	5518	703.46	-19.60	-480
W16-16	14-6	39879.916	11.454	2.783	4963	6478	715.77	3.85	480
W4-16	10-10	3829.627	4.302	2.136	7075	5998	0.00	0.00	0
W4-16	8-12	11193.323	6.665	2.234	7075	5758	192.28	54.93	-240
W4-16	12-8	6222.789	6.276	2.159	7075	6238	62.49	45.88	240
W4-16	6-14	39239.670	4.744	2.174	7075	5518	924.63	10.28	-480
W4-16	14-6	22103.821	6.230	2.031	7075	6478	477.18	44.81	480
W16-4	10-10	3820.982	7.830	2.143	7075	8110	0.00	0.00	0
W16-4	8-12	6279.254	11.264	2.217	7075	7870	64.34	43.86	-240
W16-4	12-8	10100.190	11.183	2.147	7075	8350	164.33	42.82	240
W16-4	6-14	22314.663	8.640	2.183	7075	7630	484.00	10.35	-480
W16-4	14-6	39170.225	11.276	2.152	7075	8590	925.13	44.01	480
W4-4	10-10	2756.037	4.306	1.700	9187	8110	0.00	0.00	0
W4-4	8-12	5840.800	6.030	1.645	9187	7870	111.93	40.05	-240
W4-4	12-8	5827.106	6.165	1.696	9187	8350	111.43	43.18	240
W4-4	6-14	22051.039	4.736	1.639	9187	7630	700.10	9.99	-480
W4-4	14-6	22052.330	6.245	1.708	9187	8590	700.15	45.03	480

w=4 in the lower layer reduced mean signing latency by 40.83% to 45.09% relative to placing w=4 in the upper layer.

This behavior indicates that the computational role of a WOTS parameter depends not only on its value but also on its position in the XMSSMT hierarchy. In the evaluated FAST implementation, lower-layer signing is exercised directly for each message, whereas upper-layer WOTS material is associated with the authentication of lower-layer subtree roots and retained state. As a result, placing the more expensive small-w configuration in the lower layer changes the dominant signing path differently from placing it in the upper layer.

The state-size result reinforces this asymmetry. Across all five height allocations, W4-16 required exactly 2112 B less serialized secret-key storage than W16-4. This difference is attributable to the additional retained upper-layer WOTS state when w=4 is assigned to the upper layer. Thus, the same nominal pair of Winternitz parameters can produce materially different latency and state costs depending on layer placement.

For deployment-oriented XMSSMT design, this means that heterogeneous WOTS parameterization should be treated as an ordered assignment rather than an unordered set. The configurations (4,16) and (16,4) are not interchangeable even though they contain the same two Winternitz values and produce the same signature size.

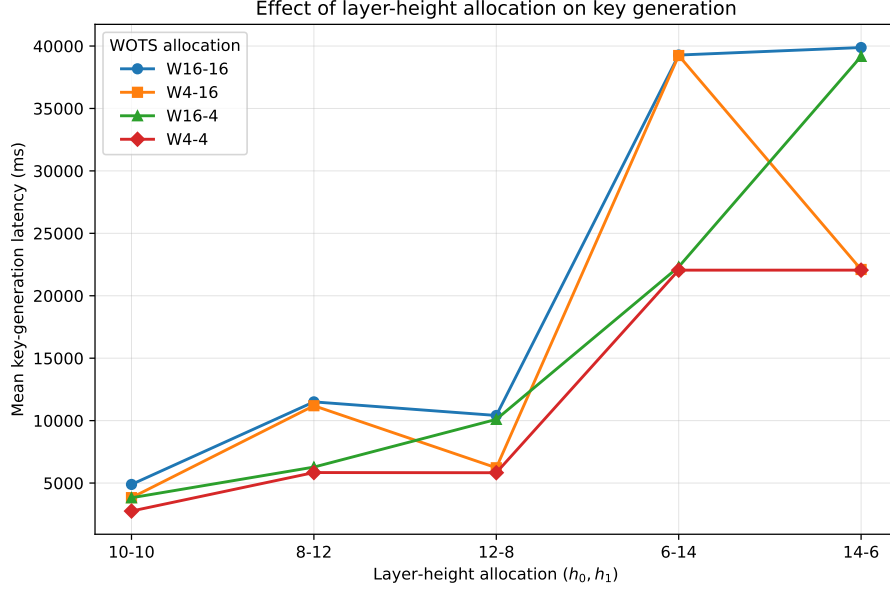


Fig. 4 Mean key-generation latency across the evaluated layer-height allocations and WOTS classes.

Table 4 Empirical Pareto frontier obtained by jointly minimizing key-generation latency, signing latency, verification latency, signature size, and serialized secret-key size. Thirteen of the 20 evaluated configurations are nondominated.

Configuration	h_0	h_1	w_0	w_1	KeyGen (ms)	Sign (ms)	Verify (ms)	Sig. (B)	SK (B)
H10-10_W16-16	10	10	16	16	4888.614	11.029	2.611	4963	5998
H10-10_W16-4	10	10	16	4	3820.982	7.830	2.143	7075	8110
H10-10_W4-16	10	10	4	16	3829.627	4.302	2.136	7075	5998
H10-10_W4-4	10	10	4	4	2756.037	4.306	1.700	9187	8110
H12-8_W16-16	12	8	16	16	10411.905	11.165	2.600	4963	6238
H12-8_W4-4	12	8	4	4	5827.106	6.165	1.696	9187	8350
H14-6_W4-16	14	6	4	16	22103.821	6.230	2.031	7075	6478
H6-14_W16-16	6	14	16	16	39277.915	8.868	2.701	4963	5518
H6-14_W4-16	6	14	4	16	39239.670	4.744	2.174	7075	5518
H6-14_W4-4	6	14	4	4	22051.039	4.736	1.639	9187	7630
H8-12_W16-16	8	12	16	16	11501.286	12.068	2.822	4963	5758
H8-12_W4-16	8	12	4	16	11193.323	6.665	2.234	7075	5758
H8-12_W4-4	8	12	4	4	5840.800	6.030	1.645	9187	7870

5.2 Height allocation and serialized state

Redistributing the fixed total height $h=20$ between the two XMSSMT layers changed serialized BDS state in a systematic manner. Relative to the balanced (10,10) allocation, moving two levels from the lower layer to the upper layer reduced serialized secret-key size by 240 B, while moving two levels in the opposite direction increased it by 240 B. A four-level redistribution produced corresponding changes of 480 B.

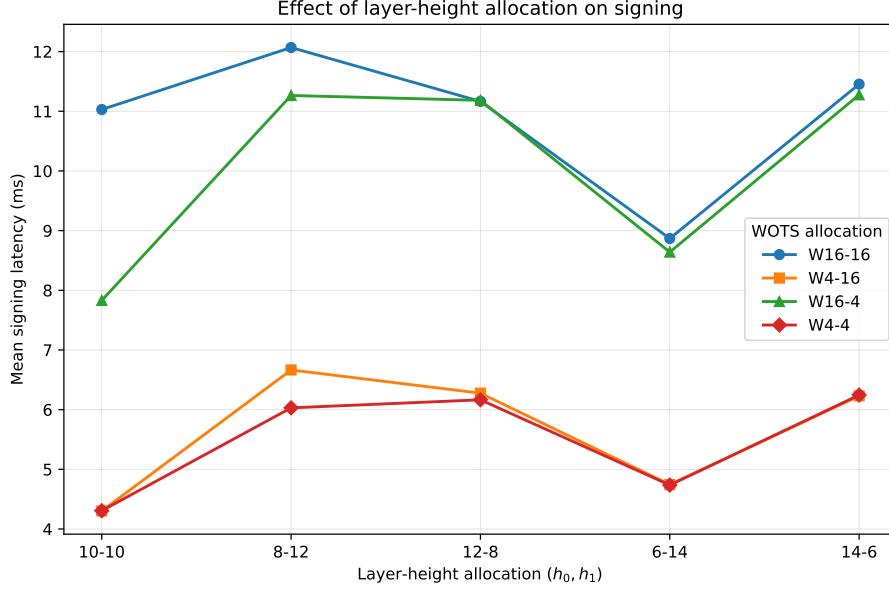


Fig. 5 Mean signing latency across the evaluated layer-height allocations and WOTS classes.

This pattern reflects the structure of the FAST implementation, where the lower layer contributes both current and next BDS states, while the upper layer contributes only its current BDS state. Increasing h_0 therefore increases serialized state more strongly than increasing h_1 by the same amount.

The latency effects, however, were not monotonic. Extreme allocations could reduce signing latency in some WOTS classes while substantially increasing key-generation time. The W16-16 comparison illustrates this trade-off: H6-14_W16-16 reduced mean signing latency from 11.029 ms to 8.868 ms and reduced serialized secret-key size from 5998 B to 5518 B, but increased key-generation latency from 4888.614 ms to 39277.915 ms.

This indicates that height redistribution should not be treated as a simple latency optimization. Its effect depends on the interaction between subtree sizes, BDS-state construction, layer-specific WOTS behavior, and the operation being optimized.

5.3 Multi-objective design implications

The Pareto analysis identified 13 nondominated configurations out of the 20 evaluated parameter combinations. This relatively broad nondominated set is expected because five independent objectives were minimized simultaneously: key-generation latency, signing latency, verification latency, signature size, and serialized secret-key size.

Different configurations occupied different empirical performance roles. H10-10_W4-16 achieved the minimum observed signing latency of 4.302 ms. H10-10_W4-4 achieved the minimum key-generation latency of 2756.037 ms. H6-14_W4-4 achieved the minimum verification latency of 1.639 ms. W16-16 configurations retained the

minimum signature size of 4963 B, while H6-14_W16-16 and H6-14_W4-16 achieved the minimum serialized secret-key size of 5518 B.

These results show why a single scalar ranking would obscure important deployment trade-offs. A configuration optimized for frequent signing may not be desirable for systems that prioritize compact signatures or fast key initialization. Likewise, reducing serialized state may be beneficial for constrained storage but can coincide with substantial key-generation cost.

Accordingly, heterogeneous XMSSMT configuration is a multi-objective parameter-selection problem rather than a search for a universal optimum.

5.4 Practical interpretation of the balanced W4-16 configuration

Among the tested configurations, H10-10_W4-16 is particularly informative because it combines the balanced height allocation with asymmetric WOTS placement. It achieved the minimum mean signing latency of 4.302 ms while retaining a 5998-B serialized secret key.

Compared with H10-10_W16-4, which produces the same 7075-B signature size, H10-10_W4-16 reduced signing latency from 7.830 ms to 4.302 ms and reduced serialized secret-key size from 8110 B to 5998 B. Key-generation latencies were also similar: 3829.627 ms for H10-10_W4-16 and 3820.982 ms for H10-10_W16-4; verification latencies were likewise close.

This comparison isolates a useful design principle in the evaluated implementation: when an asymmetric WOTS allocation is acceptable, assigning the smaller Winternitz parameter to the lower layer can provide a favorable signing/state trade-off without increasing signature size relative to the reverse placement.

This observation is an empirical result for the tested implementation and parameter grid, rather than a claim of universal optimality.

5.5 Experimental and methodological limitations

Several limitations constrain the interpretation of the results.

First, the heterogeneous configurations are experimental extensions and are not standardized XMSSMT parameter sets. The study therefore evaluates implementation-level design alternatives rather than proposing replacement standardized OIDs.

Second, the 20-cell comparison combines measurements preserved from the HEIGHT, WOTS, and JOINT experimental phases. Although each source passed its own correctness, benchmark, thermal, and integrity gates, the complete 20-cell matrix was not measured as a single session-matched factorial experiment. The joint Pareto surface should therefore be interpreted as an empirical comparison of frozen datasets.

Third, the H6-14 configurations cross the lower-layer rollover boundary between signature indices 63 and 64 within the 100-signature measurement sequence. Their mean signing latency consequently includes this state-transition event, whereas configurations with larger lower-layer heights do not encounter an equivalent rollover within the same 100-signature window.

Fourth, the experiments were conducted on one Raspberry Pi 5 platform using one compiler/toolchain configuration and the FAST BDS implementation. Absolute latency values and some relative effects may differ on other processors, compilers, traversal implementations, or hash-function backends.

Finally, the study evaluates only five height allocations and four WOTS allocations. The conclusions therefore apply to the tested design space and should not be interpreted as an exhaustive search over all possible heterogeneous XMSSMT parameterizations.

5.6 Research contribution

The study demonstrates that XMSSMT layer parameters can be treated as heterogeneous design variables rather than being constrained to homogeneous per-layer assignments. The experimental framework supports independent layer-specific Winternitz parameters and tree heights while preserving the total XMSSMT height.

Three empirical contributions follow from this framework.

First, the experiments establish a consistent WOTS-placement effect: lower-layer placement of $w=4$ reduced signing latency by 40.83% to 45.09% compared with upper-layer placement at equal signature size, while also reducing serialized secret-key state by 2112 B.

Second, the height-allocation experiments quantify the asymmetric contribution of the two layers to serialized BDS state and show that state reduction can trade against substantially higher key-generation latency.

Third, the joint 20-configuration analysis demonstrates that heterogeneous XMSSMT parameter selection is intrinsically multi-objective, with 13 nondominated configurations under the evaluated five-objective model.

Together, these results provide an implementation-level basis for deployment-oriented XMSSMT parameter selection under explicit latency, signature-size, and state-storage constraints.

6 Conclusion

This study evaluated layer-specific XMSSMT parameterization in a two-layer FAST/BDS implementation on Raspberry Pi 5. Rather than treating heterogeneous XMSSMT parameters as a new theoretical construction, the work focused on their implementation-level behavior and deployment trade-offs.

The experiments combined five layer-height allocations with four ordered WOTS allocations, producing a 20-configuration design space. The results demonstrate that parameter placement is not interchangeable across XMSSMT layers. In particular, the ordered WOTS allocations (4,16) and (16,4) produced the same 7075-byte signature size but exhibited substantially different signing and state costs. Placing $w=4$ in the lower layer reduced mean signing latency by 40.83% to 45.09% across the five tested height allocations and reduced serialized secret-key state by 2112 bytes in every comparison.

Layer-height redistribution also produced systematic state effects. Relative to the balanced (10,10) allocation, the tested asymmetric configurations changed serialized

secret-key size by -240, +240, -480, or +480 bytes. These state reductions did not translate into uniformly lower latency: extreme height allocations could improve signing latency while imposing large key-generation penalties. The results therefore show that tree-height allocation should be treated as a multi-objective design variable rather than a single-metric optimization.

The five-objective Pareto analysis further demonstrated this trade-off structure. Of the 20 evaluated configurations, 13 were nondominated when minimizing key-generation latency, signing latency, verification latency, signature size, and serialized secret-key size. No single configuration minimized all objectives, supporting deployment-dependent parameter selection instead of a universal parameter recommendation.

The findings provide an empirical basis for treating XMSSMT layer parameters as ordered implementation variables whose effects depend on their position in the hierarchy. For constrained systems, this creates opportunities to tune signing latency, serialized state, signature size, and initialization cost according to application requirements.

The conclusions are limited to the tested FAST/BDS implementation, Raspberry Pi 5 platform, compiler configuration, and parameter grid. The heterogeneous configurations evaluated here are experimental and are not standardized XMSSMT parameter sets or standardized OIDs. Future work should extend the evaluation to additional Winternitz values, deeper XMSSMT hierarchies, alternative hash backends, different processor classes, and session-matched full-factorial benchmarking to determine how broadly the observed placement effects generalize.

Acknowledgements. The authors acknowledge the support of the School of Computer Sciences, Universiti Sains Malaysia, and the Cybersecurity Research Centre (CYRES).

Declarations

Author contributions. Sohaib Rana: Conceptualization, methodology, software, validation, formal analysis, investigation, data curation, visualization, and writing – original draft.

Mohd Najwadi Yusoff: Supervision, conceptualization, methodology, project administration, and writing – review and editing.

Je Sen Teh: Supervision, methodology, validation, and writing – review and editing.

Adnan Anwar: Validation and writing – review and editing.

Funding. The authors received no specific funding for this work.

Data availability. The benchmark datasets, normalized operation-level measurements, analysis outputs, tables, and integrity manifests generated for this study are available from the corresponding author on reasonable request.

Code availability. The implementation source, build metadata, and reproducibility artifacts used in this study are available from the corresponding author on reasonable request.

Competing interests. The authors declare that they have no competing interests.

References

- [1] Hülsing, A., Butin, D., Gazdag, S.-L., Rijneveld, J., Mohaisen, A.: XMSS: extended merkle signature scheme. RFC 8391, Internet Research Task Force (May 2018). <https://doi.org/10.17487/RFC8391>
- [2] Cooper, D.A., Apon, D.C., Dang, Q.H., Davidson, M.S., Dworkin, M.J., Miller, C.A.: Recommendation for stateful hash-based signature schemes. NIST Special Publication 800-208, National Institute of Standards and Technology (October 2020). <https://doi.org/10.6028/NIST.SP.800-208>
- [3] Hülsing, A., Rausch, L., Buchmann, J.: Optimal parameters for XMSSMT. In: Security Engineering and Intelligence Informatics. Lecture Notes in Computer Science, vol. 8128, pp. 194–208. Springer, ??? (2013). https://doi.org/10.1007/978-3-642-40588-4_14
- [4] Buchmann, J., Dahmen, E., Schneider, M.: Merkle tree traversal revisited. In: Post-Quantum Cryptography. Lecture Notes in Computer Science, vol. 5299, pp. 63–78. Springer, ??? (2008). https://doi.org/10.1007/978-3-540-88403-3_5
- [5] Wang, W., Jungk, B., Wälde, J., Deng, S., Gupta, N., Szefer, J., Niederhagen, R.: XMSS and Embedded Systems – XMSS Hardware Accelerators for RISC-V. IACR Cryptology ePrint Archive **2018**, 1225 (2018). Report 2018/1225
- [6] Campos, F., Kohlstadt, T., Reith, S., Stöttinger, M.: LMS vs XMSS: Comparison of stateful hash-based signature schemes on ARM Cortex-M4. In: Progress in Cryptology – AFRICACRYPT 2020. Lecture Notes in Computer Science, vol. 12174, pp. 258–277. Springer, ??? (2020). https://doi.org/10.1007/978-3-030-51938-4_13
- [7] Bos, J.W., Hülsing, A., Renes, J., Vredendaal, C.: Rapidly verifiable XMSS signatures. IACR Transactions on Cryptographic Hardware and Embedded Systems **2021**(1), 137–168 (2021) <https://doi.org/10.46586/tches.v2021.i1.137-168>
- [8] Thoma, J.P., Güneysu, T.: A configurable hardware implementation of XMSS. IACR Cryptology ePrint Archive **2021**, 352 (2021). Report 2021/352